

TÍNH ĐA HÌNH TRONG JAVA (COMPILE-TIME VÀ RUNTIME)

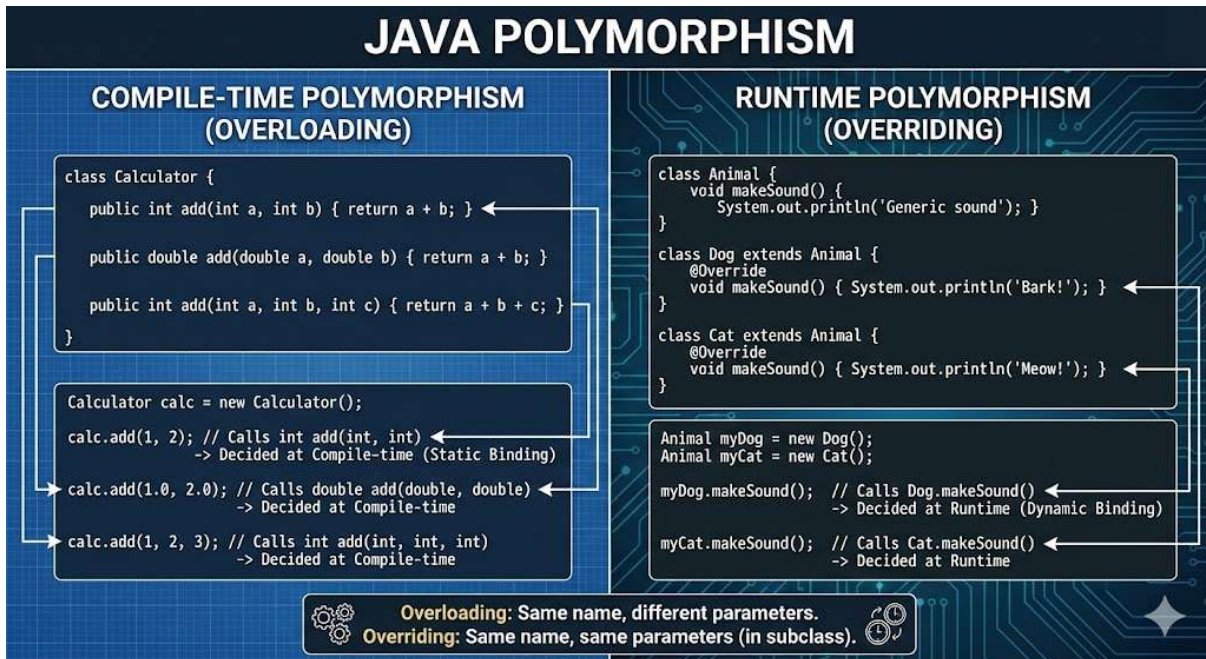
1. Tổng quan về Đa hình (Polymorphism)

Đa hình (Polymorphism) là một trong bốn nguyên lý cốt lõi của Lập trình hướng đối tượng (OOP). Thuật ngữ này xuất phát từ tiếng Hy Lạp, mang ý nghĩa là "nhiều hình thái".

Trong ngữ cảnh của Java, tính đa hình cho phép một hành động có thể thực hiện theo nhiều cách khác nhau tùy thuộc vào đối tượng đang thực hiện hành động đó. Điều này giúp mã nguồn trở nên linh hoạt, dễ mở rộng và giảm sự phụ thuộc chặt chẽ giữa các thành phần.

Java chia đa hình thành hai loại chính:

- Đa hình tại thời điểm biên dịch (Compile-time Polymorphism):** Thực hiện thông qua cơ chế Nạp chồng (Overloading).
- Đa hình tại thời điểm chạy (Runtime Polymorphism):** Thực hiện thông qua cơ chế Ghi đè (Overriding).



2. Đa hình tại thời điểm biên dịch (Compile-time Polymorphism)

Loại đa hình này còn được gọi là **Liên kết tĩnh (Static Binding)** hoặc **Liên kết sớm (Early Binding)**.

2.1. Cơ chế hoạt động

Đa hình biên dịch được thực hiện thông qua kỹ thuật **Nạp chồng phương thức (Method Overloading)**.

Khi mã nguồn được biên dịch, trình biên dịch (Compiler) sẽ xác định chính xác phương thức nào cần được gọi dựa trên **chữ ký phương thức** (tên phương thức và danh sách tham số). Vì quyết định này được đưa ra ngay lúc biên dịch (trước khi chương trình chạy), nên nó được gọi là đa hình tại thời điểm biên dịch.

2.2. Ví dụ minh họa

Xem xét lớp **TinhToan** dưới đây với phương thức **cong** được nạp chồng:

```

class TinhToan {
    // Phương thức cộng 2 số nguyên
    public int cong(int a, int b) {
        return a + b;
    }

    // Phương thức cộng 2 số thực (Nạp chồng)
    public double cong(double a, double b) {
        return a + b;
    }

    // Phương thức cộng 3 số nguyên (Nạp chồng)
    public int cong(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        TinhToan obj = new TinhToan();

        // Trình biên dịch biết ngay cần gọi phương thức nào dựa vào tham số
        obj.cong(10, 20); // Gọi cong(int, int)
        obj.cong(10.5, 20.5); // Gọi cong(double, double)
    }
}

```

Trong ví dụ trên, trình biên dịch nhìn vào đối số (10, 20) và biết ngay nó khớp với `cong(int, int)`. Không có sự mơ hồ nào xảy ra khi chạy chương trình.

3. Đa hình tại thời điểm chạy (Runtime Polymorphism)

Loại đa hình này còn được gọi là **Liên kết động (Dynamic Binding)** hoặc **Liên kết muộn (Late Binding)**.

3.1. Cơ chế hoạt động

Đa hình runtime được thực hiện thông qua kỹ thuật **Ghi đè phương thức (Method Overriding)** kết hợp với **Upcasting** (Ép kiểu lên).

Trong trường hợp này, phương thức được gọi không được quyết định bởi trình biên dịch. Thay vào đó, Máy ảo Java (JVM) sẽ quyết định gọi phương thức nào **trong quá trình chương trình đang chạy** (runtime), dựa trên **đối tượng thực tế** mà biến tham chiếu đang trỏ tới, chứ không phải dựa trên kiểu của biến tham chiếu.

3.2. Điều kiện tiên quyết

Để có đa hình runtime, cần thỏa mãn 3 điều kiện:

1. Phải có quan hệ kế thừa (Inheritance).
2. Phải có ghi đè phương thức (Overriding).
3. Phải sử dụng tham chiếu của lớp cha để trỏ tới đối tượng của lớp con (Upcasting).

3.3. Ví dụ minh họa

```

// Lớp cha
class DongVat {
    void tiengKeu() {
        System.out.println("Động vật phát ra âm thanh...");
    }
}

// Lớp con 1
class Cho extends DongVat {
    @Override
    void tiengKeu() {
        System.out.println("Gâu gâu");
    }
}

// Lớp con 2
class Meo extends DongVat {
    @Override
    void tiengKeu() {
        System.out.println("Meo meo");
    }
}

public class Main {
    public static void main(String[] args) {
        // Biến tham chiếu là DongVat (Cha), nhưng đối tượng thực là Cho (Con)
        DongVat a = new Cho(); // Upcasting

        // Tại thời điểm biên dịch: Trình biên dịch chỉ biết 'a' là DongVat.
        // Tại thời điểm chạy: JVM thấy 'a' đang trỏ tới đối tượng 'Cho'.
        // -> Kết quả: Gọi phương thức tiengKeu() của lớp Cho.
        a.tiengKeu(); // In ra: Gâu gâu

        // Thay đổi tham chiếu sang đối tượng Mèo
        a = new Meo();
        a.tiengKeu(); // In ra: Meo meo
    }
}

```

Đây là sức mạnh của đa hình runtime: Cùng một dòng lệnh `a.tiengKeu()` nhưng kết quả lại khác nhau tùy thuộc vào việc `a` đang là `Cho` hay `Meo`.

4. So sánh Compile-time và Runtime Polymorphism

Bảng dưới đây tóm tắt sự khác biệt cơ bản giữa hai loại đa hình này:

Tiêu chí	Compile-time Polymorphism	Runtime Polymorphism
Kỹ thuật	Nạp chồng phương thức (Overloading).	Ghi đè phương thức (Overriding).

Cơ chế liên kết	Liên kết tĩnh (Static Binding).	Liên kết động (Dynamic Binding).
Thời điểm xử lý	Được giải quyết bởi Compiler khi biên dịch.	Được giải quyết bởi JVM khi chương trình chạy.
Hiệu năng	Nhanh hơn (do biết trước phương thức cần gọi).	Chậm hơn đôi chút (do cần kiểm tra đối tượng lúc chạy).
Độ linh hoạt	Kém linh hoạt hơn.	Rất linh hoạt, hỗ trợ mở rộng hệ thống tốt.
Yêu cầu kế thừa	Không bắt buộc.	Bắt buộc phải có quan hệ kế thừa.

5. Kết luận

Tính đa hình là yếu tố then chốt giúp Java trở thành một ngôn ngữ mạnh mẽ.

- **Compile-time Polymorphism (Overloading)** giúp mã nguồn gọn gàng, dễ đọc và tiện dụng cho lập trình viên.
- **Runtime Polymorphism (Overriding)** là nền tảng để xây dựng các hệ thống lỏng lẻo (loose coupling), nơi mà các thành phần có thể dễ dàng thay thế và mở rộng mà không làm ảnh hưởng đến cấu trúc tổng thể của chương trình.